Writing the Main Application Logic in routes.py

Milestone 3 - Activity 3.1

1. Overview

Milestone 3 focuses on **routes.py**, the file that connects everything together. It defines every URL the application responds to, captures the user's input, calls the AI functions built in Milestone 2, and passes the results to the HTML templates (or returns them as JSON). In the project workflow this is **Milestone 3 - Activity 3.1: Writing the Main Application Logic in routes.py**, between the core functionalities (Milestone 2) and the frontend (Milestone 4).

2. Where routes.py fits

The backend is organised so that each file does one job. routes.py is the coordinator that calls the others:

Function	File	What it does
generate_outline()	gemini_flash.py	Uses Gemini Flash to create a structured 5-panel outline. Each panel has a panel number, title, scene description and image prompt. Output: list of dictionaries.
generate_story()	gemini_pro.py	Uses Gemini Pro to expand the outline into narration and character dialogue. Output: one formatted text with all panel stories.
generate_image()	image_generator.py	Uses Stable Diffusion to create an image from a prompt, sanitizes the prompt for a safe filename, saves to static/panels and returns the file path.
build_comic_layout()	layout_builder.py	Matches each image with its panel story. Output: list of dictionaries with panel number, image path and text.
save_pdf()	exporters.py	Builds a multi-page PDF with FPDF, saves it in static/exports and returns the path.

3. The core routes

Separate routes were created for each of ComicCraft's core features:

Route	Purpose
`/`	Loads the **homepage** (index.html) where users submit their story details.
`/generate`	Handles the **form submission**, processes the input using AI, generates the comic panels and returns the **comic preview** page (comic_preview.html).
`/generate-comic/json`	An **API route** that accepts a JSON payload, runs the same generation flow and returns the comic layout data and the generated PDF path.
`/export-success`	Shows a **success confirmation page** (export_success.html) after the comic is downloaded.

Route	Purpose
`/test-image`	A **developer utility** route that tests image generation from a direct prompt, without creating a full comic.

Each route is a separate entry point: browser users use the form, while developers and API clients use the JSON endpoint.

4. Handling user input

Route handlers capture data in two different ways depending on how the request is sent:

a) HTML form input - /generate

The form in index.html sends five fields. In the route they are read with FastAPI's `Form(...)` parameters (this is why the `python-multipart` package is installed):

- Story prompt
- Character name
- Setting (school, forest, space, city)
- Story tone (light-hearted, dramatic, poetic, funny)
- Art style (anime, pixel art, comic book, realistic)

b) JSON input - /generate-comic/json

For API clients, the JSON body is validated automatically with a **Pydantic** model called `PromptRequest`. If a field is missing or has the wrong type, FastAPI rejects the request before your code runs.

5. AI workflow inside the routes

Both /generate and /generate-comic/json follow the same five-step pipeline:

Step	Call	Result
1	generate_outline()	5-panel structured story outline (Gemini Flash)
2	generate_story()	Narration and dialogues (Gemini Pro)
3	generate_image() - once per panel	One comic-style image for each panel (Stable Diffusion)
4	build_comic_layout()	Final panel layout: number, image path, text
5	save_pdf()	Exportable PDF file in static/exports

After step 5, /generate sends the data to **comic_preview.html** through Jinja2, while /generate-comic/json returns the layout data and PDF path as a JSON response.

Flow diagram

```
User form  ->  POST /generate
                    |
                    v
          generate_outline()   (Gemini Flash)
                    |
          generate_story()     (Gemini Pro)
```

```
                |
        generate_image() x panels (Stable Diffusion)
                |
        build_comic_layout()
                |
        save_pdf()
                |
                v
    comic_preview.html  (Jinja2)   -->  Download PDF  -->  /export-success
```

6. Error handling

- Route handlers use FastAPI's `HTTPException` to return clear HTTP errors to the client.
- `try / except` blocks catch unexpected failures such as Gemini API errors or JSON decode errors, so the app fails gracefully instead of crashing.
- Form and JSON data are extracted and passed correctly into the backend functions, so AI processing always receives the user's details.

7. Illustrative skeleton

The code below is a simplified **example** to show the structure of routes.py. Your actual file (shown as screenshots in the project document) may differ in details such as variable names.

```
from fastapi import APIRouter, Request, Form, HTTPException
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel

router = APIRouter()
templates = Jinja2Templates(directory="templates")

class PromptRequest(BaseModel):
    prompt: str
    character: str
    setting: str
    tone: str
    art_style: str

@router.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@router.post("/generate", response_class=HTMLResponse)
async def generate(request: Request,
                   prompt: str = Form(...), character: str = Form(...),
                   setting: str = Form(...), tone: str = Form(...),
                   art_style: str = Form(...)):
    try:
        outline = generate_outline(prompt, character, setting, tone, art_style)
        story   = generate_story(outline)
        images  = [generate_image(p["image_prompt"]) for p in outline]
        layout  = build_comic_layout(images, story)
        pdf     = save_pdf(layout)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
    return templates.TemplateResponse("comic_preview.html",
                                      {"request": request, "panels": layout, "pdf": pdf})
```

8. Summary checklist for Activity 3.1

- All five routes defined: /, /generate, /generate-comic/json, /export-success, /test-image.
- Form fields captured with Form(...); JSON validated with PromptRequest.
- generate_outline, generate_story, generate_image, build_comic_layout and save_pdf integrated in the generation routes.
- Results passed to comic_preview.html and export_success.html, or returned as JSON.
- Errors handled with HTTPException and try-except.